talk08 练习与作业

目录

0.1 练习和作业说明

将相关代码填写入以 “‘{r} “‘ 标志的代码框中，运行并看到正确的结果；

完成后，用工具栏里的”Knit” 按键生成 PDF 文档；

将 PDF 文档改为：姓名`-`学号`-talk08` 作业`.pdf`，并提交到老师指定的平台/钉群。

0.2 talk08 内容回顾

- for loop
- `apply` functions
- `dplyr` 的本质是遍历
- `map` functions in `purrr` package

0.3 练习与作业：用户验证

请运行以下命令，验证你的用户名。

如你当前用户名不能体现你的真实姓名，请改为拼音后再运行本作业！

```
Sys.info()[["user"]]
```

```
## [1] "weihuachen"
```

```
Sys.getenv("HOME")
```

```
## [1] "/Users/weihuachen"
```

0.4 练习与作业 1：loop 初步

0.4.1 loop 练习（部分内容来自 r-exercises.com 网站）

1. 写一个循环，计算从 1 到 7 的平方并打印 `print`；
2. 取 `iris` 的列名，计算每个列名的长度，并打印为下面的格式：`Sepal.Length (12)`；
3. 写一个 `while` 循环，每次用 `rnorm` 取一个随机数字并打印，直到取到的数字大于 1；
4. 写一个循环，计算 Fibonacci 序列的值超过 1 百万所需的循环数；注：Fibonacci 序列的规则为：`0, 1, 1, 2, 3, 5, 8, 13, 21 ...`；

```
## 代码写这里，并运行；
for (i in 1:7) {
  print(i ^ 2)
}
```

```
## [1] 1
## [1] 4
## [1] 9
## [1] 16
## [1] 25
## [1] 36
## [1] 49
```

```
for (nm in colnames(iris)) {
  cat(nm, "(", nchar(nm), ")\n", sep = "")
}
```

```
## Sepal.Length(12)
## Sepal.Width(11)
## Petal.Length(12)
## Petal.Width(11)
## Species(7)
```

```
set.seed(1)
x <- rnorm(1)
while (x <= 1) {
  print(x)
  x <- rnorm(1)
}
```

```
## [1] -0.6264538
## [1] 0.1836433
## [1] -0.8356286
```

```
print(x)
```

```
## [1] 1.595281
```

```
a<- 0
b<- 1
count <- 1
while (b <= 1e6) {
```

```
  tmp<- b
  b<- a + b
  a<- tmp
  count<- count + 1
}

count
```

```
## [1] 31
```

```
b
```

```
## [1] 1346269
```

0.5 练习与作业 2：loop 进阶，系统和其它函数

0.5.1 生成一个数字 matrix，并做练习

生成一个 100 x 100 的数字 matrix：

1. 行、列平均，用 rowMeans, colMeans 函数；
2. 行、列平均，用 apply 函数
3. 行、列总和，用 rowSums, colSums 函数；
4. 行、列总和，用 apply 函数
5. 使用自定义函数，同时计算：
    - 行平均、总和、sd
    - 列平均、总和、sd

```
## 代码写这里，并运行；
set.seed(123)
m <- matrix(
  rnorm(100 * 100),
  nrow = 100,
```

```
  ncol = 100
)
rowMeans(m)
```

```
##   [1] -0.026684225 -0.083114303 -0.183423561 -0.111917400  0.039644460
##   [6]  0.043346195 -0.024307629  0.051188520  0.040623072  0.088355476
##  [11]  0.104363897  0.100739618  0.013541027  0.052171152 -0.046877341
##  [16] -0.089008784  0.052389217 -0.196002357 -0.003666222  0.049000961
##  [21] -0.039859892  0.006463048  0.100712343  0.033092742 -0.100764852
##  [26]  0.004456002  0.188762744 -0.070139611 -0.011071747 -0.019174355
##  [31]  0.060749487 -0.063456344 -0.152788583 -0.215180127 -0.090059409
##  [36] -0.094145372  0.070200733  0.044794666  0.042297432  0.020154504
##  [41]  0.020063831 -0.007340383 -0.019883849  0.105729016 -0.008198700
##  [46] -0.216123309  0.049000391 -0.099369559  0.061816145 -0.124865417
##  [51] -0.021397978  0.056231663  0.116910793  0.109304134  0.062710166
##  [56] -0.041424779  0.091383202  0.126511160  0.003273445 -0.064581385
##  [61]  0.156667345  0.024305139  0.051429639  0.115650277 -0.158779618
##  [66]  0.019436928  0.050436460  0.054598434  0.090908651 -0.019283572
##  [71] -0.030236447 -0.171216818  0.019279322  0.003953032 -0.115156776
##  [76]  0.143417369  0.206472690 -0.128927097  0.012674240 -0.016486735
##  [81]  0.006606047  0.070514364 -0.049961024 -0.028892001 -0.136064071
##  [86]  0.056870001 -0.149181317 -0.057067347  0.020194095 -0.034540678
##  [91] -0.043990843 -0.067004317  0.200569724  0.040386570  0.007603851
##  [96] -0.026311988 -0.019585358 -0.050763357  0.005624061 -0.076472826
```

```
colMeans(m)
```

```
##   [1]  0.090405909 -0.107546798  0.120465110 -0.036222908  0.105850925
##   [6] -0.042299958 -0.149644141  0.105873547  0.093589714 -0.019192740
##  [11]  0.119940576 -0.019175558 -0.002539253 -0.060211278  0.092989267
##  [16]  0.035486952  0.089034057  0.031054735  0.065184175  0.072888861
##  [21] -0.118428579 -0.132873799 -0.103876010  0.073030345 -0.038059336
##  [26]  0.106087594 -0.114998513 -0.015194869  0.047547311  0.095640577
##  [31] -0.053892068 -0.154321040  0.038796044  0.051143836  0.005222176
```

```
##  [36]  0.067957430 -0.017637086 -0.249917976  0.072493960  0.148550961
##  [41] -0.083001504 -0.042175162 -0.012272628 -0.041267127 -0.055456214
##  [46]  0.061739186  0.059723192 -0.067013442 -0.053320694 -0.088637443
##  [51]  0.068030935  0.035564772 -0.078207500 -0.035319963  0.100236830
##  [56]  0.081086225 -0.022973924  0.013780474  0.036581643  0.146011054
##  [61]  0.051491656 -0.237923602  0.081151956 -0.168992567  0.111133042
##  [66]  0.154474925 -0.092308075 -0.163588773  0.046865402 -0.111339105
##  [71]  0.050670448 -0.049291862  0.162929984 -0.005236446  0.107308791
##  [76] -0.146529425  0.095533720 -0.039136175  0.054898411 -0.176539282
##  [81] -0.045987198 -0.005408580  0.128398513  0.154826990  0.015862826
##  [86]  0.081288428 -0.048529871  0.024177534  0.109537307 -0.214803458
##  [91]  0.006974302 -0.061529427 -0.058303417 -0.092120691  0.055597849
##  [96] -0.062518360 -0.010375903 -0.091063377 -0.130560897 -0.034516638
```

```
apply(m, 1, mean)
```

```
##   [1] -0.026684225 -0.083114303 -0.183423561 -0.111917400  0.039644460
##   [6]  0.043346195 -0.024307629  0.051188520  0.040623072  0.088355476
##  [11]  0.104363897  0.100739618  0.013541027  0.052171152 -0.046877341
##  [16] -0.089008784  0.052389217 -0.196002357 -0.003666222  0.049000961
##  [21] -0.039859892  0.006463048  0.100712343  0.033092742 -0.100764852
##  [26]  0.004456002  0.188762744 -0.070139611 -0.011071747 -0.019174355
##  [31]  0.060749487 -0.063456344 -0.152788583 -0.215180127 -0.090059409
##  [36] -0.094145372  0.070200733  0.044794666  0.042297432  0.020154504
##  [41]  0.020063831 -0.007340383 -0.019883849  0.105729016 -0.008198700
##  [46] -0.216123309  0.049000391 -0.099369559  0.061816145 -0.124865417
##  [51] -0.021397978  0.056231663  0.116910793  0.109304134  0.062710166
##  [56] -0.041424779  0.091383202  0.126511160  0.003273445 -0.064581385
##  [61]  0.156667345  0.024305139  0.051429639  0.115650277 -0.158779618
##  [66]  0.019436928  0.050436460  0.054598434  0.090908651 -0.019283572
##  [71] -0.030236447 -0.171216818  0.019279322  0.003953032 -0.115156776
##  [76]  0.143417369  0.206472690 -0.128927097  0.012674240 -0.016486735
##  [81]  0.006606047  0.070514364 -0.049961024 -0.028892001 -0.136064071
##  [86]  0.056870001 -0.149181317 -0.057067347  0.020194095 -0.034540678
```

```
##  [91] -0.043990843 -0.067004317  0.200569724  0.040386570  0.007603851
##  [96] -0.026311988 -0.019585358 -0.050763357  0.005624061 -0.076472826
```

```
apply(m, 2, mean)
```

```
##   [1]  0.090405909 -0.107546798  0.120465110 -0.036222908  0.105850925
##   [6] -0.042299958 -0.149644141  0.105873547  0.093589714 -0.019192740
##  [11]  0.119940576 -0.019175558 -0.002539253 -0.060211278  0.092989267
##  [16]  0.035486952  0.089034057  0.031054735  0.065184175  0.072888861
##  [21] -0.118428579 -0.132873799 -0.103876010  0.073030345 -0.038059336
##  [26]  0.106087594 -0.114998513 -0.015194869  0.047547311  0.095640577
##  [31] -0.053892068 -0.154321040  0.038796044  0.051143836  0.005222176
##  [36]  0.067957430 -0.017637086 -0.249917976  0.072493960  0.148550961
##  [41] -0.083001504 -0.042175162 -0.012272628 -0.041267127 -0.055456214
##  [46]  0.061739186  0.059723192 -0.067013442 -0.053320694 -0.088637443
##  [51]  0.068030935  0.035564772 -0.078207500 -0.035319963  0.100236830
##  [56]  0.081086225 -0.022973924  0.013780474  0.036581643  0.146011054
##  [61]  0.051491656 -0.237923602  0.081151956 -0.168992567  0.111133042
##  [66]  0.154474925 -0.092308075 -0.163588773  0.046865402 -0.111339105
##  [71]  0.050670448 -0.049291862  0.162929984 -0.005236446  0.107308791
##  [76] -0.146529425  0.095533720 -0.039136175  0.054898411 -0.176539282
##  [81] -0.045987198 -0.005408580  0.128398513  0.154826990  0.015862826
##  [86]  0.081288428 -0.048529871  0.024177534  0.109537307 -0.214803458
##  [91]  0.006974302 -0.061529427 -0.058303417 -0.092120691  0.055597849
##  [96] -0.062518360 -0.010375903 -0.091063377 -0.130560897 -0.034516638
```

```
rowSums(m)
```

```
##   [1]  -2.6684225  -8.3114303 -18.3423561 -11.1917400   3.9644460   4.3346195
##   [7]  -2.4307629   5.1188520   4.0623072   8.8355476  10.4363897  10.0739618
##  [13]   1.3541027   5.2171152  -4.6877341  -8.9008784   5.2389217 -19.6002357
##  [19]  -0.3666222   4.9000961  -3.9859892   0.6463048  10.0712343   3.3092742
##  [25] -10.0764852   0.4456002  18.8762744  -7.0139611  -1.1071747  -1.9174355
##  [31]   6.0749487  -6.3456344 -15.2788583 -21.5180127  -9.0059409  -9.4145372
##  [37]   7.0200733   4.4794666   4.2297432   2.0154504   2.0063831  -0.7340383
```

```
##  [43]  -1.9883849  10.5729016  -0.8198700 -21.6123309   4.9000391  -9.9369559
##  [49]   6.1816145 -12.4865417  -2.1397978   5.6231663  11.6910793  10.9304134
##  [55]   6.2710166  -4.1424779   9.1383202  12.6511160   0.3273445  -6.4581385
##  [61]  15.6667345   2.4305139   5.1429639  11.5650277 -15.8779618   1.9436928
##  [67]   5.0436460   5.4598434   9.0908651  -1.9283572  -3.0236447 -17.1216818
##  [73]   1.9279322   0.3953032 -11.5156776  14.3417369  20.6472690 -12.8927097
##  [79]   1.2674240  -1.6486735   0.6606047   7.0514364  -4.9961024  -2.8892001
##  [85] -13.6064071   5.6870001 -14.9181317  -5.7067347   2.0194095  -3.4540678
##  [91]  -4.3990843  -6.7004317  20.0569724   4.0386570   0.7603851  -2.6311988
##  [97]  -1.9585358  -5.0763357   0.5624061  -7.6472826
```

```
colSums(m)
```

```
##   [1]   9.0405909 -10.7546798  12.0465110  -3.6222908  10.5850925  -4.2299958
##   [7] -14.9644141  10.5873547   9.3589714  -1.9192740  11.9940576  -1.9175558
##  [13]  -0.2539253  -6.0211278   9.2989267   3.5486952   8.9034057   3.1054735
##  [19]   6.5184175   7.2888861 -11.8428579 -13.2873799 -10.3876010   7.3030345
##  [25]  -3.8059336  10.6087594 -11.4998513  -1.5194869   4.7547311   9.5640577
##  [31]  -5.3892068 -15.4321040   3.8796044   5.1143836   0.5222176   6.7957430
##  [37]  -1.7637086 -24.9917976   7.2493960  14.8550961  -8.3001504  -4.2175162
##  [43]  -1.2272628  -4.1267127  -5.5456214   6.1739186   5.9723192  -6.7013442
##  [49]  -5.3320694  -8.8637443   6.8030935   3.5564772  -7.8207500  -3.5319963
##  [55]  10.0236830   8.1086225  -2.2973924   1.3780474   3.6581643  14.6011054
##  [61]   5.1491656 -23.7923602   8.1151956 -16.8992567  11.1133042  15.4474925
##  [67]  -9.2308075 -16.3588773   4.6865402 -11.1339105   5.0670448  -4.9291862
##  [73]  16.2929984  -0.5236446  10.7308791 -14.6529425   9.5533720  -3.9136175
##  [79]   5.4898411 -17.6539282  -4.5987198  -0.5408580  12.8398513  15.4826990
##  [85]   1.5862826   8.1288428  -4.8529871   2.4177534  10.9537307 -21.4803458
##  [91]   0.6974302  -6.1529427  -5.8303417  -9.2120691   5.5597849  -6.2518360
##  [97]  -1.0375903  -9.1063377 -13.0560897  -3.4516638
```

```
apply(m, 1, sum)
```

```
##   [1]  -2.6684225  -8.3114303 -18.3423561 -11.1917400   3.9644460   4.3346195
##   [7]  -2.4307629   5.1188520   4.0623072   8.8355476  10.4363897  10.0739618
```

```
##  [13]   1.3541027   5.2171152  -4.6877341  -8.9008784   5.2389217 -19.6002357
##  [19]  -0.3666222   4.9000961  -3.9859892   0.6463048  10.0712343   3.3092742
##  [25] -10.0764852   0.4456002  18.8762744  -7.0139611  -1.1071747  -1.9174355
##  [31]   6.0749487  -6.3456344 -15.2788583 -21.5180127  -9.0059409  -9.4145372
##  [37]   7.0200733   4.4794666   4.2297432   2.0154504   2.0063831  -0.7340383
##  [43]  -1.9883849  10.5729016  -0.8198700 -21.6123309   4.9000391  -9.9369559
##  [49]   6.1816145 -12.4865417  -2.1397978   5.6231663  11.6910793  10.9304134
##  [55]   6.2710166  -4.1424779   9.1383202  12.6511160   0.3273445  -6.4581385
##  [61]  15.6667345   2.4305139   5.1429639  11.5650277 -15.8779618   1.9436928
##  [67]   5.0436460   5.4598434   9.0908651  -1.9283572  -3.0236447 -17.1216818
##  [73]   1.9279322   0.3953032 -11.5156776  14.3417369  20.6472690 -12.8927097
##  [79]   1.2674240  -1.6486735   0.6606047   7.0514364  -4.9961024  -2.8892001
##  [85] -13.6064071   5.6870001 -14.9181317  -5.7067347   2.0194095  -3.4540678
##  [91]  -4.3990843  -6.7004317  20.0569724   4.0386570   0.7603851  -2.6311988
##  [97]  -1.9585358  -5.0763357   0.5624061  -7.6472826
```

```
apply(m, 2, sum)
```

```
##   [1]   9.0405909 -10.7546798  12.0465110  -3.6222908  10.5850925  -4.2299958
##   [7] -14.9644141  10.5873547   9.3589714  -1.9192740  11.9940576  -1.9175558
##  [13]  -0.2539253  -6.0211278   9.2989267   3.5486952   8.9034057   3.1054735
##  [19]   6.5184175   7.2888861 -11.8428579 -13.2873799 -10.3876010   7.3030345
##  [25]  -3.8059336  10.6087594 -11.4998513  -1.5194869   4.7547311   9.5640577
##  [31]  -5.3892068 -15.4321040   3.8796044   5.1143836   0.5222176   6.7957430
##  [37]  -1.7637086 -24.9917976   7.2493960  14.8550961  -8.3001504  -4.2175162
##  [43]  -1.2272628  -4.1267127  -5.5456214   6.1739186   5.9723192  -6.7013442
##  [49]  -5.3320694  -8.8637443   6.8030935   3.5564772  -7.8207500  -3.5319963
##  [55]  10.0236830   8.1086225  -2.2973924   1.3780474   3.6581643  14.6011054
##  [61]   5.1491656 -23.7923602   8.1151956 -16.8992567  11.1133042  15.4474925
##  [67]  -9.2308075 -16.3588773   4.6865402 -11.1339105   5.0670448  -4.9291862
##  [73]  16.2929984  -0.5236446  10.7308791 -14.6529425   9.5533720  -3.9136175
##  [79]   5.4898411 -17.6539282  -4.5987198  -0.5408580  12.8398513  15.4826990
##  [85]   1.5862826   8.1288428  -4.8529871   2.4177534  10.9537307 -21.4803458
##  [91]   0.6974302  -6.1529427  -5.8303417  -9.2120691   5.5597849  -6.2518360
```

```
##  [97]  -1.0375903  -9.1063377 -13.0560897  -3.4516638
```

```
row_stats <- apply( m, 1,
  function(v) c(
    mean = mean(v),
    sum  = sum(v),
    sd   = sd(v)
  )
)
row_stats
```

```
##             [,1]       [,2]        [,3]        [,4]       [,5]       [,6]
## mean -0.02668423 -0.0831143  -0.1834236  -0.1119174 0.03964446 0.04334619
## sum  -2.66842255 -8.3114303 -18.3423561 -11.1917400 3.96444598 4.33461946
## sd    0.95896881  0.9371213   1.0357881   0.9826430 1.21433472 0.97225711
##             [,7]       [,8]       [,9]      [,10]      [,11]      [,12]
## mean -0.02430763 0.05118852 0.04062307 0.08835548  0.1043639  0.1007396
## sum  -2.43076286 5.11885198 4.06230725 8.83554758 10.4363897 10.0739618
## sd    0.95427426 1.04649415 0.94548228 0.95083753  1.0073338  1.0131549
##            [,13]      [,14]       [,15]       [,16]      [,17]       [,18]
## mean 0.01354103 0.05217115 -0.04687734 -0.08900878 0.05238922  -0.1960024
## sum  1.35410275 5.21711516 -4.68773405 -8.90087838 5.23892166 -19.6002357
## sd   0.95494410 0.89000381  0.99246671  1.04712673 0.95595318   1.0799562
##             [,19]      [,20]       [,21]       [,22]      [,23]      [,24]
## mean -0.003666222 0.04900096 -0.03985989 0.006463048  0.1007123 0.03309274
## sum  -0.366622198 4.90009613 -3.98598921 0.646304780 10.0712343 3.30927421
## sd    0.939138831 1.05476229  0.97463882 0.998988654  1.0466785 0.98009455
##            [,25]       [,26]      [,27]       [,28]       [,29]       [,30]
## mean  -0.1007649 0.004456002  0.1887627 -0.07013961 -0.01107175 -0.01917436
## sum  -10.0764852 0.445600243 18.8762744 -7.01396111 -1.10717468 -1.91743551
## sd     0.8418984 1.055241346  0.9836935  0.99691235  1.11135296  0.96959222
##            [,31]       [,32]       [,33]       [,34]       [,35]       [,36]
## mean 0.06074949 -0.06345634  -0.1527886  -0.2151801 -0.09005941 -0.09414537
## sum  6.07494871 -6.34563444 -15.2788583 -21.5180127 -9.00594087 -9.41453715
```

```
## sd   1.00109580  1.02654189   0.9789725   1.0527243  0.97587133  1.04725528
##           [,37]       [,38]       [,39]      [,40]       [,41]        [,42]
## mean 0.07020073 0.04479467 0.04229743 0.0201545 0.02006383 -0.007340383
## sum  7.02007325 4.47946663 4.22974320 2.0154504 2.00638315 -0.734038298
## sd   1.03573246 0.90408621 1.04484359 0.9028340 1.03313424  0.864168949
##            [,43]      [,44]       [,45]        [,46]       [,47]        [,48]
## mean -0.01988385  0.105729 -0.0081987  -0.2161233 0.04900039 -0.09936956
## sum  -1.98838491 10.572902 -0.8198700 -21.6123309 4.90003905 -9.93695594
## sd    0.96629053  1.067903  0.9676976   1.0511907 1.05191109  0.99489321
##           [,49]       [,50]       [,51]       [,52]       [,53]       [,54]
## mean 0.06181614  -0.1248654 -0.02139798 0.05623166  0.1169108  0.1093041
## sum  6.18161449 -12.4865417 -2.13979778 5.62316633 11.6910793 10.9304134
## sd   1.11684686   1.0449273  1.04022211 0.99815837  1.0169544  1.0211720
##           [,55]       [,56]      [,57]       [,58]       [,59]       [,60]
## mean 0.06271017 -0.04142478 0.0913832  0.1265112 0.003273445 -0.06458139
## sum  6.27101660 -4.14247789 9.1383202 12.6511160 0.327344504 -6.45813854
## sd   0.96474514  1.08286756 1.0657764  0.9091267 0.952980204  1.03612451
##           [,61]       [,62]       [,63]       [,64]       [,65]       [,66]
## mean  0.1566673 0.02430514 0.05142964  0.1156503  -0.1587796 0.01943693
## sum  15.6667345 2.43051394 5.14296386 11.5650277 -15.8779618 1.94369283
## sd    1.0989584 0.91885544 0.98216722  1.0927986   1.0788178 0.96783522
##           [,67]       [,68]       [,69]       [,70]       [,71]       [,72]
## mean 0.05043646 0.05459843 0.09090865 -0.01928357 -0.03023645  -0.1712168
## sum  5.04364601 5.45984340 9.09086508 -1.92835724 -3.02364468 -17.1216818
## sd   0.92909244 0.86845520 0.90276666  1.00195768  0.95726548   1.0668083
##           [,73]       [,74]       [,75]       [,76]       [,77]       [,78]
## mean 0.01927932 0.003953032  -0.1151568  0.1434174  0.2064727  -0.1289271
## sum  1.92793218 0.395303236 -11.5156776 14.3417369 20.6472690 -12.8927097
## sd   0.97806855 0.973162790   0.9576052  1.1094790  0.9202278   1.0168183
##           [,79]       [,80]       [,81]       [,82]       [,83]      [,84]
## mean 0.01267424 -0.01648673 0.006606047 0.07051436 -0.04996102 -0.028892
## sum  1.26742397 -1.64867349 0.660604736 7.05143638 -4.99610241 -2.889200
## sd   1.00315617  1.06558283 0.950500895 0.94065649  1.03449034  1.047337
```

```
##            [,85]    [,86]       [,87]       [,88]      [,89]       [,90]
## mean  -0.1360641 0.056870  -0.1491813 -0.05706735 0.02019409 -0.03454068
## sum  -13.6064071 5.687000 -14.9181317 -5.70673474 2.01940945 -3.45406778
## sd     0.9956054 1.016281   1.0593105  1.03721315 1.04902106  0.93404306
##            [,91]       [,92]      [,93]      [,94]       [,95]       [,96]
## mean -0.04399084 -0.06700432  0.2005697 0.04038657 0.007603851 -0.02631199
## sum  -4.39908435 -6.70043165 20.0569724 4.03865699 0.760385096 -2.63119877
## sd    1.03878935  0.97678270  0.9553286 0.93340325 1.077681313  1.10261280
##            [,97]       [,98]       [,99]      [,100]
## mean -0.01958536 -0.05076336 0.005624061 -0.07647283
## sum  -1.95853580 -5.07633575 0.562406083 -7.64728261
## sd    0.93840089  0.85434195 0.890020632  0.91854618
```

```
col_stats <- apply( m, 2,
  function(v) c(
    mean = mean(v),
    sum  = sum(v),
    sd   = sd(v)
  )
)
col_stats
```

```
##            [,1]        [,2]       [,3]        [,4]       [,5]        [,6]
## mean 0.09040591  -0.1075468  0.1204651 -0.03622291  0.1058509 -0.04229996
## sum  9.04059086 -10.7546798 12.0465110 -3.62229084 10.5850925 -4.22999578
## sd   0.91281588   0.9669866  0.9498790  1.03878122  0.9893458  0.93872815
##             [,7]       [,8]       [,9]       [,10]      [,11]       [,12]
## mean  -0.1496441  0.1058735 0.09358971 -0.01919274  0.1199406 -0.01917556
## sum  -14.9644141 10.5873547 9.35897144 -1.91927403 11.9940576 -1.91755583
## sd     1.0282366  1.0100100 1.05180659  1.02033166  1.0429982  0.99767147
##             [,13]       [,14]      [,15]      [,16]      [,17]      [,18]
## mean -0.002539253 -0.06021128 0.09298927 0.03548695 0.08903406 0.03105473
## sum  -0.253925262 -6.02112781 9.29892666 3.54869516 8.90340573 3.10547347
## sd    0.928007043  1.01382919 0.98614286 0.88929622 1.09648466 1.02443405
```

```
##           [,19]      [,20]       [,21]       [,22]       [,23]      [,24]
## mean 0.06518418 0.07288886  -0.1184286  -0.1328738  -0.1038760 0.07303035
## sum  6.51841753 7.28888607 -11.8428579 -13.2873799 -10.3876010 7.30303453
## sd   1.04354899 1.08710170   0.8867420   0.8671375   0.9234582 1.01178395
##           [,25]      [,26]       [,27]       [,28]      [,29]      [,30]
## mean -0.03805934  0.1060876  -0.1149985 -0.01519487 0.04754731 0.09564058
## sum  -3.80593362 10.6087594 -11.4998513 -1.51948694 4.75473105 9.56405766
## sd    0.99456268  0.9751193   1.0359941  0.94454890 1.01726182 1.10530160
##           [,31]       [,32]      [,33]      [,34]       [,35]      [,36]
## mean -0.05389207  -0.1543210 0.03879604 0.05114384 0.005222176 0.06795743
## sum  -5.38920679 -15.4321040 3.87960435 5.11438357 0.522217582 6.79574304
## sd    1.07991180   0.8705622 1.07626589 0.84481290 1.047729024 1.07836057
##           [,37]       [,38]      [,39]     [,40]      [,41]       [,42]
## mean -0.01763709  -0.2499180 0.07249396  0.148551 -0.0830015 -0.04217516
## sum  -1.76370864 -24.9917976 7.24939600 14.855096 -8.3001504 -4.21751621
## sd    0.97329530   0.8768267 0.91526278  1.095703  1.1023174  0.78330286
##           [,43]       [,44]       [,45]      [,46]      [,47]       [,48]
## mean -0.01227263 -0.04126713 -0.05545621 0.06173919 0.05972319 -0.06701344
## sum  -1.22726277 -4.12671273 -5.54562141 6.17391864 5.97231916 -6.70134424
## sd    1.01742000  1.08455734  1.00191901 1.00882076 0.96523497  0.98094922
##           [,49]       [,50]      [,51]      [,52]      [,53]       [,54]
## mean -0.05332069 -0.08863744 0.06803094 0.03556477 -0.0782075 -0.03531996
## sum  -5.33206943 -8.86374425 6.80309354 3.55647721 -7.8207500 -3.53199625
## sd    0.94231314  1.11142003 0.88940618 0.86387437  0.9641162  1.00713013
##           [,55]      [,56]       [,57]      [,58]      [,59]      [,60]
## mean  0.1002368 0.08108622 -0.02297392 0.01378047 0.03658164  0.1460111
## sum  10.0236830 8.10862250 -2.29739242 1.37804736 3.65816433 14.6011054
## sd    1.0771408 1.01709562  1.01292952 0.98644219 1.02524416  1.0675179
##           [,61]       [,62]      [,63]       [,64]      [,65]      [,66]
## mean 0.05149166  -0.2379236 0.08115196  -0.1689926  0.1111330  0.1544749
## sum  5.14916559 -23.7923602 8.11519559 -16.8992567 11.1133042 15.4474925
## sd   0.93443052   1.1540099 1.04262975   1.0431884  0.9165161  0.9676328
##           [,67]       [,68]     [,69]       [,70]      [,71]       [,72]
```

```
## mean -0.09230807  -0.1635888 0.0468654  -0.1113391 0.05067045 -0.04929186
## sum  -9.23080748 -16.3588773 4.6865402 -11.1339105 5.06704482 -4.92918617
## sd    0.87696277   1.0257226 0.9645486   0.9469142 1.01823607  0.87801873
##           [,73]        [,74]      [,75]       [,76]      [,77]       [,78]
## mean  0.1629300 -0.005236446  0.1073088  -0.1465294 0.09553372 -0.03913617
## sum  16.2929984 -0.523644574 10.7308791 -14.6529425 9.55337199 -3.91361747
## sd    0.8941483  1.008655635  0.9944261   1.0769782 0.96638316  1.01031030
##           [,79]       [,80]      [,81]       [,82]      [,83]     [,84]
## mean 0.05489841  -0.1765393 -0.0459872 -0.00540858  0.1283985  0.154827
## sum  5.48984112 -17.6539282 -4.5987198 -0.54085797 12.8398513 15.482699
## sd   1.05630156   1.0659721  1.0161120  1.07501409  0.9959257  0.969051
##           [,85]      [,86]       [,87]      [,88]      [,89]       [,90]
## mean 0.01586283 0.08128843 -0.04852987 0.02417753  0.1095373  -0.2148035
## sum  1.58628257 8.12884279 -4.85298714 2.41775337 10.9537307 -21.4803458
## sd   1.11207550 1.06293318  0.94756707 1.04856646  1.0506158   1.1474328
##            [,91]       [,92]       [,93]       [,94]      [,95]       [,96]
## mean 0.006974302 -0.06152943 -0.05830342 -0.09212069 0.05559785 -0.06251836
## sum  0.697430160 -6.15294266 -5.83034172 -9.21206910 5.55978494 -6.25183599
## sd   0.994143844  0.98719987  1.03118035  0.83965976 1.06511882  0.98403060
##           [,97]       [,98]       [,99]      [,100]
## mean -0.0103759 -0.09106338  -0.1305609 -0.03451664
## sum  -1.0375903 -9.10633772 -13.0560897 -3.45166380
## sd    0.9922376  0.91410182   1.0321389  0.99787848
```

0.5.2 用 mtcars 进行练习

用 tapply 练习:

1. 用 **汽缸数**分组，计算 **油耗**的 **平均值**;
2. 用 **汽缸数**分组，计算 **wt** 的 **平均值**;

用 dplyr 的函数实现上述计算

```r
## 代码写这里，并运行；
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
tapply(mtcars$mpg, mtcars$cyl, mean)
```

```
##        4        6        8
## 26.66364 19.74286 15.10000
```

```r
tapply(mtcars$wt,  mtcars$cyl, mean)
```

```
##        4        6        8
## 2.285727 3.117143 3.999214
```

```r
mtcars %>%
  group_by(cyl) %>%
  summarise(
    mean_mpg = mean(mpg),
    mean_wt  = mean(wt),
    .groups  = "drop"
  )
```

```
## # A tibble: 3 x 3
##     cyl mean_mpg mean_wt
##   <dbl>    <dbl>   <dbl>
## 1     4     26.7    2.29
## 2     6     19.7    3.12
```

```
## 3     8     15.1    4.00
```

0.5.3 练习 lapply 和 sapply

1. 分别用 lapply 和 sapply 计算下面 list 里每个成员 vector 的长度:

```
list( a = 1:10, b = letters[1:5], c = LETTERS[1:8] );
```

2. 分别用 lapply 和 sapply 计算 mtcars 每列的平均值;

```
## 代码写这里，并运行;
lst <- list(
  a = 1:10,
  b = letters[1:5],
  c = LETTERS[1:8]
)

lapply(lst, length)
```

```
## $a
## [1] 10
##
## $b
## [1] 5
##
## $c
## [1] 8
```

```
sapply(lst, length)
```

```
##  a  b  c
## 10  5  8
```

```r
lapply(mtcars, mean)
```

```
## $mpg
## [1] 20.09062
##
## $cyl
## [1] 6.1875
##
## $disp
## [1] 230.7219
##
## $hp
## [1] 146.6875
##
## $drat
## [1] 3.596563
##
## $wt
## [1] 3.21725
##
## $qsec
## [1] 17.84875
##
## $vs
## [1] 0.4375
##
## $am
## [1] 0.40625
##
## $gear
## [1] 3.6875
##
## $carb
```

```
## [1] 2.8125
```

```
sapply(mtcars, mean)
```

```
##        mpg        cyl       disp         hp       drat         wt       qsec
##  20.090625   6.187500 230.721875 146.687500   3.596563   3.217250  17.848750
##         vs         am       gear       carb
##   0.437500   0.406250   3.687500   2.812500
```

0.6 练习与作业 3：loop 进阶，purr 包的函数

0.6.1 map 初步

生成一个变量：

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
```

用 map 计算：

- 列平均值、总和和中值

```
## 代码写这里，并运行；
library(purrr)
library(tibble)
set.seed(123)

df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
```

```
  c = rnorm(10),
  d = rnorm(10)
)

map(df, mean)
```

```
## $a
## [1] 0.07462564
##
## $b
## [1] 0.208622
##
## $c
## [1] -0.4245589
##
## $d
## [1] 0.3220446
```

```
map(df, sum)
```

```
## $a
## [1] 0.7462564
##
## $b
## [1] 2.08622
##
## $c
## [1] -4.245589
##
## $d
## [1] 3.220446
```

```
map(df, median)
```

```
## $a
```

```
## [1] -0.07983455
##
## $b
## [1] 0.3802926
##
## $c
## [1] -0.6769652
##
## $d
## [1] 0.4901909
```

```
map_dbl(df, mean)
```

```
##           a           b           c           d
##  0.07462564  0.20862196 -0.42455887  0.32204455
```

```
map_dbl(df, sum)
```

```
##          a          b          c          d
##  0.7462564  2.0862196 -4.2455887  3.2204455
```

```
map_dbl(df, median)
```

```
##           a           b           c           d
## -0.07983455  0.38029264 -0.67696525  0.49019094
```

0.6.2 map 进阶

用 map 配合 purr 包中其它函数，用 mtcars：

为每一个 **汽缸数**计算燃油效率 mpg 与重量 wt 的相关性（Pearson correlation），得到 p 值和 correlation coefficient 值。

```
## 代码写这里，并运行；
mtcars %>%
```

```
  split(.$cyl) %>%
  map(~ cor.test(~ mpg + wt, data = .x, method = "pearson")) %>%
  map(~ c(
    cor = unname(.x$estimate),
    p   = .x$p.value
  ))
```

```
## $`4`
##         cor           p
## -0.71318483  0.01374278
##
## $`6`
##         cor           p
## -0.68154982  0.09175766
##
## $`8`
##         cor           p
## -0.65035801  0.01179281
```

0.6.3 keep 和 discard

1. 保留 iris 中有 factor 的列，并打印前 10 行；
2. 去掉 iris 中有 factor 的列，并打印前 10 行；

```
## 代码写这里，并运行；
iris %>%
  keep(~ is.factor(.x)) %>%
  head(10)
```

```
##    Species
## 1   setosa
## 2   setosa
## 3   setosa
```

```
## 4   setosa
## 5   setosa
## 6   setosa
## 7   setosa
## 8   setosa
## 9   setosa
## 10  setosa
```

```
iris %>%
  discard(~ is.factor(.x)) %>%
  head(10)
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width
## 1           5.1         3.5          1.4         0.2
## 2           4.9         3.0          1.4         0.2
## 3           4.7         3.2          1.3         0.2
## 4           4.6         3.1          1.5         0.2
## 5           5.0         3.6          1.4         0.2
## 6           5.4         3.9          1.7         0.4
## 7           4.6         3.4          1.4         0.3
## 8           5.0         3.4          1.5         0.2
## 9           4.4         2.9          1.4         0.2
## 10          4.9         3.1          1.5         0.1
```

0.6.4 用 reduce

用 reduce 得到以下三个 vector 中共有的数字:

```
c(1, 3, 5, 6, 10),
  c(1, 2, 3, 7, 8, 10),
  c(1, 2, 3, 4, 8, 9, 10)
```

```
## 代码写这里，并运行；
vecs <- list(
  c(1, 3, 5, 6, 10),
  c(1, 2, 3, 7, 8, 10),
  c(1, 2, 3, 4, 8, 9, 10)
)

reduce(vecs, intersect)
```

```
## [1]  1  3 10
```

0.6.5 运行以下代码，观察得到的结果，并用 tidyverse 包中的 pivot_wider 等函数实现类似的结果

```
dfs <- list(
  age = tibble(name = "John", age = 30),
  sex = tibble(name = c("John", "Mary"), sex = c("M", "F")),
  trt = tibble(name = "Mary", treatment = "A")
);

dfs %>% reduce(full_join);
```

```
## 代码写这里，并运行；
dfs <- list(
  age = tibble(name = "John", age = 30),
  sex = tibble(name = c("John", "Mary"), sex = c("M", "F")),
  trt = tibble(name = "Mary", treatment = "A")
);

dfs %>% reduce(full_join);
```

```
## Joining with `by = join_by(name)`
```

```
## Joining with `by = join_by(name)`
```

```
## # A tibble: 2 x 4
##   name    age sex   treatment
##   <chr> <dbl> <chr> <chr>
## 1 John     30 M     <NA>
## 2 Mary     NA F     A
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.1.5
## v ggplot2   3.5.2     v stringr   1.5.1
## v lubridate 1.9.4     v tidyr     1.3.1
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
```

```
library(tidyverse)

dfs_long <- list(
  tibble(
    name  = "John",
    key   = "age",
    value = "30"
  ),
  tibble(
    name  = c("John","Mary"),
    key   = "sex",
    value = c("M","F")
  ),
  tibble(
    name  = "Mary",
    key   = "treatment",
```

```
    value = "A"
  )
) %>%
  bind_rows()

res_pivot <- dfs_long %>%
  pivot_wider(
    names_from  = key,
    values_from = value
  ) %>%
  mutate(age = as.numeric(age))

res_pivot
```

```
## # A tibble: 2 x 4
##   name    age sex   treatment
##   <chr> <dbl> <chr> <chr>
## 1 John     30 M     <NA>
## 2 Mary     NA F     A
```

0.7 练习与作业 4：pmap 和 map 的更多用法

请参考 https://r4ds.had.co.nz/iteration.html 的 Mapping over multiple arguments 部分

0.7.1 map2

运行以下代码，查看输出结果。用 for 循环重现计算结果。

```
mu <- list(5, 10, -3);
sigma <- list(1, 5, 10);
map2(mu, sigma, rnorm, n = 5)
```

```r
## 代码写这里，并运行；
set.seed(1)

mu    <- list(5, 10, -3)
sigma <- list(1, 5, 10)

out_map2 <- map2(
  mu,
  sigma,
  rnorm,
  n = 5
)
out_map2
```

```
## [[1]]
## [1] 4.373546 5.183643 4.164371 6.595281 5.329508
##
## [[2]]
## [1]  5.897658 12.437145 13.691624 12.878907  8.473058
##
## [[3]]
## [1]  12.1178117   0.8984324  -9.2124058 -25.1469989   8.2493092
```

```r
out_for <- vector("list", length(mu))

set.seed(1)
for (i in seq_along(mu)) {
  out_for[[i]] <- rnorm(
    n    = 5,
    mean = mu[[i]],
    sd   = sigma[[i]]
  )
}
out_for
```

```
## [[1]]
## [1] 4.373546 5.183643 4.164371 6.595281 5.329508
##
## [[2]]
## [1]  5.897658 12.437145 13.691624 12.878907  8.473058
##
## [[3]]
## [1]  12.1178117   0.8984324  -9.2124058 -25.1469989   8.2493092
```

0.7.2 pmap

运行以下代码，查看输出结果。用 for 循环重现计算结果。

```
params <- tribble(
  ~mean, ~sd, ~n,
    5,     1,  1,
   10,     5,  3,
   -3,    10,  5
)
params %>%
  pmap(rnorm)
```

```
## set.seed(1)

params <- tribble(
  ~mean, ~sd, ~n,
   5,     1,   1,
   10,    5,   3,
   -3,    10,  5
)

out_pmap <- params %>%
  pmap(rnorm)
```

```
out_pmap
```

```
## [[1]]
## [1] 4.955066
##
## [[2]]
## [1]  9.919049 14.719181 14.106106
##
## [[3]]
## [1]   2.939013   6.189774   4.821363  -2.254350 -22.893517
```

```
out_for2 <- vector("list", nrow(params))

set.seed(1)
for (i in seq_len(nrow(params))) {
  out_for2[[i]] <- rnorm(
    n    = params$n[i],
    mean = params$mean[i],
    sd   = params$sd[i]
  )
}
out_for2
```

```
## [[1]]
## [1] 4.373546
##
## [[2]]
## [1] 10.918217  5.821857 17.976404
##
## [[3]]
## [1]   0.2950777 -11.2046838   1.8742905   4.3832471   2.7578135
```

0.8 练习与作业 5：并行计算

0.8.1 安装相关包，成功运行以下代码，观察得到的结果，并回答问题

* parallel
* foreach
* iterators

```
library(parallel); ##
library(foreach);
```

```
##
## Attaching package: 'foreach'
```

```
## The following objects are masked from 'package:purrr':
##
##     accumulate, when
```

```
library(iterators);
( cpus <- parallel::detectCores() );
```

```
## [1] 10
```

```
d <- data.frame(x=1:10000, y=rnorm(10000));

cl <- makeCluster( cpus - 1 );

res <- foreach( row = iter( d, by = "row" ) ) %dopar% {
  return ( row$x * row$y  );
}
```

```
## Warning: executing %dopar% sequentially: no parallel backend registered
```

```
stopCluster( cl );
```

```
summary(unlist(res));
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -31135.05  -2684.99    -21.19    -41.99   2571.78  29876.88
```

问：你的系统有多少个 CPU？此次任务使用了多少个？22 个，用了 21 个
答：用代码打印出相应的数字即可：

```
## 代码写这里，并运行；
library(parallel)
library(foreach)
library(iterators)
cpus <- parallel::detectCores()
cpus
```

```
## [1] 10
```

```
cl <- makeCluster(cpus - 1)

used <- length(cl)
used
```

```
## [1] 9
```

```
stopCluster(cl)

cpus
```

```
## [1] 10
```

```
used
```

```
## [1] 9
```